Guía de desarrollo — Iker

Iker Esteban Fuentes Torres · M7 Administración y moderación · M6 Reseñas · Offero, versión del 27 de octubre de 2026

> Esta guía es tu hoja de ruta para tus módulos: qué construir, en qué orden, con qué tablas y permisos, y cómo saber que quedó bien. No trae código: el código lo escribes tú. Las historias con puntos van en el Product Backlog del plan (Parte 5). Si algo aquí choca con el documento de contexto, manda el documento y le avisas a Diego. Lo marcado con [SUPUESTO] es una propuesta que todavía hay que confirmar.

Tu resumen

Módulos	M7 Administración (complejidad media-alta: ≈30 h completo, ≈16 h mínimo) y M6 Reseñas (media-baja: ≈18 h completo, ≈10 h mínimo).
Carga estimada	≈26 h en versiones mínimas y hasta ≈48 h en completas, frente a ≈47 h netas. Tienes margen: por eso llevas el rol transversal de revisor de seguridad (RLS).
Orden de trabajo	M7 primero (Sprints 1 y 2), porque M6 depende de que existan cotizaciones aceptadas (M4, Sprint 3).
Prioridad	M6 mínimo va antes que los extras de M7. Si en la Review del Sprint 2 el equipo va atrasado, se cortan extras de M7, no M6.
Desbloqueas a	Todo el equipo (botón de reportar y moderación) y Daniel (componentes de reseñas para su página).
Dependes de	Diego (M1 y funciones de ayuda, sáb 7 nov; función de aceptar, vie 27 nov), Daniel (categories, businesses) y Wilson (requests).
Fechas clave	/administrador y reportes dom 15 nov · categorías y usuarios dom 22 nov · reseñas: tabla y formulario dom 29 nov · M6 terminado y moderación dom 6 dic.

Tu calendario por sprint

Sprint	Qué entregas
0 · 28 oct – 8 nov	Preparar tu computador; Git; JS moderno, React y App Router; SQL básico y RLS; acompañar como observador las sesiones de M1 con Diego (aprendes el contrato de autenticación que usarás en todo M7).
1 · 9 – 15 nov	Proteger /administrador; tabla reports + ReportButton (dom 15); prueba cruzada de M1 (intentar volverte admin).
2 · 16 – 22 nov	Categorías (admin) y lista de usuarios (dom 22).
3 · 23 – 29 nov	Tabla reviews + RLS + formulario (dom 29); políticas de admin para ocultar; revisar la función de aceptar de Diego.
4 · 30 nov – 6 dic	RatingSummary y ReviewList con Daniel; moderación desde reportes; M6 terminado (dom 6); extras de M7 solo si se aprobaron.
5 · 7 – 11 dic	Estabilización y revisión de seguridad final.

Ruta de aprendizaje (justo a tiempo)

Aprende cada tema justo antes de la tarea que lo necesita, no todo al principio. El orden importa: cada punto se apoya en el anterior.

Sprint 0

1. Git en equipo (Anexo C).
2. JavaScript moderno: const y let, funciones flecha, map y filter, desestructuración, async/await.
3. React y Next.js App Router.
4. SQL básico con JOIN.
5. Qué es RLS y cómo se prueba con dos cuentas de roles distintos.

Antes de M7 (Sprints 1 y 2)

1. Proteger rutas por rol con el patrón de Diego.
2. Server Actions con validación en el servidor.
3. Políticas para admin con `is_admin()`.
4. Tablas que apuntan a «cualquier cosa» (`target_type` + `target_id`) y por qué no llevan clave foránea.

Antes de M6 (Sprint 3)

1. Restricciones UNIQUE y CHECK.
2. RLS con condiciones sobre otras tablas (cotizaciones aceptadas).
3. Consultas de agregación (promedio y conteo) o vistas.

Tu rol transversal: revisor de seguridad (RLS)

- En cada PR que cree o cambie una tabla, revisa la checklist: RLS activada; políticas para SELECT, INSERT, UPDATE y DELETE; nada abierto a todos sin razón; la clave secreta nunca en el frontend.
- En la prueba cruzada de cada módulo, intenta «romper» los permisos con una cuenta de otro rol (por ejemplo, cambiando ids en la URL).
- Revisa con especial cuidado el trigger de perfiles y la función de aceptar de Diego: son funciones con privilegios elevados.
- Diego hace la aprobación final de todo lo que toca la base de datos.

Módulo 7 — Panel de administración y moderación

Objetivo	Panel /administrador, solo para admins, para gestionar categorías, consultar usuarios, recibir y resolver reportes, y ocultar o mostrar contenido.
Alcance	**Mínimo (compromiso):** proteger el panel, categorías, lista de usuarios, reportes (crear y revisar) y ocultar contenido. **Extra** (solo si se aprueba en la Review del Sprint 2): cambiar el rol de un usuario y suspender cuentas.
Fuera de alcance	Borrar cuentas, métricas avanzadas, historial completo de moderación y moderar mensajes privados.
Depende de	Diego (M1); Daniel (categories, businesses); Wilson (requests); tus propias reseñas (M6) para moderarlas.
Desbloquea a	Botón «Reportar» para todo el equipo.

Tabla `reports`

Campo	Tipo sugerido	Reglas
id	uuid	Clave primaria, automática.
reporter_id	uuid → profiles.id	Quien reporta: el usuario actual.
target_type	text	business, request o review (CHECK) [SUPUESTO: los mensajes privados quedan fuera del MVP].
target_id	uuid	Id del elemento reportado. Sin clave foránea, porque puede ser de tres tablas distintas.
reason	text	Obligatorio, 10 a 500 caracteres.
status	text	open, reviewed o dismissed; por defecto open.
created_at	timestamptz	Automático.
Única (reporter_id, target_type, target_id)	—	Un usuario reporta el mismo elemento una sola vez.

Políticas RLS de `reports`

Operación	Quién	Condición
SELECT	Usuario, admin	El que reportó ve los suyos; el admin ve todos.
INSERT	Usuario con sesión	`reporter_id` es el usuario actual; status open.
UPDATE	Admin	Cambiar status.
DELETE	Nadie	—

Políticas de admin en tablas de otros

Van en tu propio archivo SQL y las revisa el dueño de cada tabla antes de fusionar.

Tabla	Dueño	Permiso del admin
`categories`	Daniel	Crear, renombrar y borrar (acuerden quién escribe la política).
`businesses`	Daniel	Cambiar `is_hidden`.
`requests`	Wilson	Cambiar `is_hidden`.
`reviews`	Iker	Cambiar `is_hidden`.
`profiles`	Diego	Cambiar `role` (extra).

Borrar una categoría en uso debe fallar, porque la clave foránea no tiene borrado en cascada. Muestra un mensaje claro: «Categoría en uso; renómbrala en vez de borrarla».

Rutas y archivos

- `src/app/administrador/page.js` — resumen: reportes abiertos y conteos. Protegido en el servidor con `is_admin()`; además las políticas bloquean todo aunque alguien llame a Supabase directo.
- `src/app/administrador/categorias/page.js`, `reportes/page.js` y `usuarios/page.js` [SUPUESTO: subrutas nuevas].
- `src/components/ui/ReportButton.js` — botón «Reportar» reutilizable: Daniel lo pone en negocios, Wilson en solicitudes y tú en reseñas.
- `src/services/admin.js` y `src/services/reports.js`.

Pasos en orden (cada paso es un pull request pequeño)

#	Paso	Fecha meta	Cómo verificar que quedó bien
1	Proteger /administrador con el patrón de Diego.	mar 10 nov	Una cuenta no admin es redirigida; la de Diego (admin) entra.
2	Tabla reports + RLS + ReportButton + Server Action.	dom 15 nov	Un reporte repetido no se duplica.
3	Categorías: lista, crear, renombrar y borrar si no está en uso (con Daniel).	jue 19 nov	Una categoría nueva aparece en el filtro de negocios.
4	Lista de usuarios: nombre, rol y fecha de registro.	dom 22 nov	Solo el admin ve la lista.
5	Reportes: lista, ver el elemento reportado, marcar revisado o descartado.	mié 25 nov	El estado cambia y queda registrado.
6	Ocultar o mostrar contenido (políticas de admin en tablas ajenas).	jue 3 dic	Un negocio oculto desaparece del listado público.
7	Extra (solo si se aprobó en la Review del Sprint 2): cambiar el rol de un usuario.	—	—
8	Prueba cruzada (la hace Diego) y publicación.	dom 6 dic	Checklist de la Definition of Done completo (Anexo G).

Criterios de aceptación

- **Dado** un usuario que no es admin, **cuando** abre /administrador o llama directo a una operación de admin, **entonces** es redirigido o la operación falla.
- **Dado** un admin, **cuando** crea una categoría, **entonces** aparece en el filtro de negocios y en el formulario de solicitudes.
- **Dado** un admin, **cuando** intenta borrar una categoría en uso, **entonces** el sistema lo impide y explica por qué.
- **Dado** un usuario, **cuando** reporta una reseña con un motivo, **entonces** el reporte aparece como abierto en el panel; si reporta lo mismo otra vez, no se duplica.
- **Dado** un admin, **cuando** oculta un negocio reportado, **entonces** deja de aparecer en los listados públicos, pero el dueño y el admin todavía lo ven.

Errores típicos y cómo evitarlos

- Proteger el panel solo ocultando el enlace del menú.
- Escribir políticas de admin en tablas ajenas sin avisar al dueño de la tabla.
- Mostrar correos de usuarios fuera del panel de admin.
- Borrar contenido en vez de ocultarlo: se pierde el historial y se rompen relaciones.

Coordinación con el equipo

- Daniel: acuerden quién escribe la política de admin en categories; él coloca ReportButton en el detalle del negocio.
- Wilson: coloca ReportButton en el detalle de la solicitud.
- Diego: hace la prueba cruzada de M7 y aprueba tus políticas sobre tablas ajenas.

Módulo 6 — Reseñas y valoraciones

Objetivo	El cliente que aceptó una cotización califica al negocio de 1 a 5 con un comentario, una vez por solicitud; la página del negocio muestra el promedio y las reseñas.
Alcance	**Mínimo (compromiso):** crear la reseña, mostrar promedio y lista, ocultar desde el admin. **Completo:** el autor edita su reseña durante 7 días [SUPUESTO] y la lista se ordena por fecha o puntuación.
Fuera de alcance	Respuesta del negocio a la reseña, fotos y reseñas de clientes hechas por proveedores.
Depende de	Diego (M4: cotizaciones aceptadas, vie 27 nov); Wilson (detalle de la solicitud, donde va el formulario); Daniel (detalle del negocio, donde se muestran las reseñas).
Desbloquea a	Nadie: es un módulo final.

Regla contra reseñas falsas [SUPUESTO; decisión pendiente de la sección 16]: solo reseña el cliente de una solicitud adjudicada, al negocio cuya cotización fue aceptada, una sola vez por solicitud. Opción más estricta para debatir: exigir que el cliente marque «trabajo completado» (requiere un estado nuevo en requests; fuera del MVP).

Tabla `reviews`

Campo	Tipo sugerido	Reglas
id	uuid	Clave primaria, automática.
reviewer_id	uuid → profiles.id	El cliente que reseña: el usuario actual.
business_id	uuid → businesses.id	El negocio reseñado.
request_id	uuid → requests.id	Única: una reseña por solicitud.
rating	integer	Entre 1 y 5 (CHECK).
comment	text	Obligatorio, 10 a 1000 caracteres [SUPUESTO].
is_hidden	boolean	Falso por defecto; solo el admin lo cambia.
created_at	timestamptz	Automático.

Políticas RLS de `reviews`

Operación	Quién	Condición
SELECT	Todos	Las no ocultas; el autor y el admin ven también las ocultas.
INSERT	Cliente	`reviewer_id` es el usuario actual; es el cliente de `request_id`; existe una cotización accepted de `business_id` para esa solicitud.
UPDATE	Admin (mínimo)	Solo `is_hidden`. En la versión completa, el autor edita rating y comment durante 7 días.
DELETE	Nadie	—

Promedio: una vista (por ejemplo `business_ratings`, con business_id, promedio y cantidad) o una consulta de agregación en tu servicio. Debe excluir las reseñas ocultas. Si usas vista, revisa con Diego que no exponga datos de más.

Rutas y archivos

- `src/components/reviews/ReviewForm.js` — Wilson lo coloca en el detalle de la solicitud cuando está adjudicada, el usuario es el dueño y aún no reseñó.
- `src/components/reviews/RatingSummary.js` y `ReviewList.js` — Daniel los coloca en el detalle del negocio; cada reseña lleva ReportButton.
- `src/services/reviews.js`.

Pasos en orden (cada paso es un pull request pequeño)

#	Paso	Fecha meta	Cómo verificar que quedó bien
1	Tabla reviews + RLS en dev (usa cotizaciones aceptadas de prueba en dev).	mié 25 nov	Un usuario sin cotización aceptada no puede insertar.
2	ReviewForm + Server Action con validaciones.	dom 29 nov	Una segunda reseña de la misma solicitud se rechaza.
3	RatingSummary y ReviewList; integración con Daniel.	mié 2 dic	El promedio coincide con un cálculo hecho a mano.
4	Prueba cruzada (la hace Wilson) y publicación con una reseña real de una transacción real del piloto.	dom 6 dic	Checklist de la Definition of Done completo (Anexo G).

> **Ojo con la regla de «sin datos ficticios»:** para marcar M6 como Terminado necesitas al menos una reseña real, y eso exige que una empresa del piloto haya aceptado una cotización real. Coordínalo con Diego (PO) desde el Sprint 3.

Criterios de aceptación

- **Dado** un cliente cuya solicitud se adjudicó al negocio A, **cuando** deja 4 estrellas y un comentario, **entonces** la reseña aparece en la página de A y el promedio se actualiza.
- **Dado** el mismo cliente, **cuando** intenta reseñar otra vez la misma solicitud, **entonces** el sistema lo impide.
- **Dado** un cliente con la solicitud todavía abierta, o un usuario que nunca contrató a A, **cuando** intenta reseñar a A desde la app o directo contra Supabase, **entonces** la operación falla.
- **Dado** un admin, **cuando** oculta una reseña, **entonces** deja de mostrarse y de contar en el promedio.

Errores típicos y cómo evitarlos

- Validar la regla de reseñas solo en el formulario.
- Un promedio que incluye reseñas ocultas.
- Guardar la puntuación como texto.

Coordinación con el equipo

- Diego: avísale cuando tengas la tabla; necesitas su función de aceptar (vie 27 nov) para probar con datos reales.
- Wilson: coloca ReviewForm en el detalle de la solicitud adjudicada.
- Daniel: coloca RatingSummary y ReviewList en el detalle del negocio.

Anexo — Acuerdos comunes del equipo

Este anexo es igual en las cuatro guías. Si cambia, se cambia en todas.

A. Calendario del equipo

Sprint	Fechas	Meta del equipo
0	mié 28 oct – dom 8 nov	Proyecto publicado (M0), entornos listos, todos con su computador preparado, M1 en marcha
1	lun 9 – dom 15 nov	M1 terminado; tablas categories, businesses, requests y reports creadas con RLS
2	lun 16 – dom 22 nov	M2 y M3 terminados; envío de cotizaciones; panel de admin básico
3	lun 23 – dom 29 nov	M4 terminado: solicitud → cotización → elegir, demostrable con usuarios reales
4	lun 30 nov – dom 6 dic	M5 y M6 (mínimos) terminados; M7 mínimo terminado
5	lun 7 – vie 11 dic	Estabilización: pruebas, errores y presentación. Entrega: vie 11 dic

Festivos en Colombia dentro del periodo: lun 2 nov, lun 16 nov y mar 8 dic.

B. Preparar tu computador (Windows, PowerShell)

#	Paso	Comando o acción	Cómo verificar
1	Instalar Node.js LTS	`winget install OpenJS.NodeJS.LTS` (o el instalador LTS de nodejs.org)	`node -v` y `npm -v` muestran versión
2	Instalar Git	`winget install Git.Git`	`git --version`
3	Permitir scripts de npm	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (responde S)	`npx -v` funciona sin el error «scripts deshabilitados»

4	Configurar Git	`git config --global user.name "Tu Nombre"`
		`git config --global user.email "correo de tu GitHub"`
		`git config --global init.defaultBranch main`
			`git config --global --list`

5	Extensiones de VS Code	ESLint, Tailwind CSS IntelliSense y Prettier; opcional: GitLens	Aparecen en la pestaña Extensiones

6	Clonar fuera de OneDrive	`mkdir C:\dev`
		`cd C:\dev`
		`git clone https://github.com/diegodani2030/App-Offero.git`
			Existe `C:\dev\App-Offero`

7	Instalar dependencias (cuando M0 esté en main)	`cd C:\dev\App-Offero` `npm install`	Termina sin errores en rojo
8	Variables de entorno	`Copy-Item .env.example .env.local` y llena los valores que Diego te envía por mensaje privado	`.env.local` existe y `git status` no lo muestra
9	Correr la app	`npm run dev` y abre http://localhost:3000	Ves la portada de Offero

Abre una PowerShell nueva después de instalar Node y Git. El correo de Git debe ser el de tu cuenta de GitHub. La primera vez que hagas push, Windows abrirá el navegador para iniciar sesión en GitHub.

C. Flujo de Git paso a paso

#	Qué haces	Comando o acción
1	Toma una tarea del tablero y muévela a «En curso»	GitHub Projects
2	Actualiza main	`git switch main` → `git pull`
3	Crea tu rama: tipo/mN-descripcion (tipos: feat, fix, chore, docs)	`git switch -c feat/m3-formulario-solicitud`
4	Guarda avances pequeños con mensajes en español: tipo(módulo): qué hace	`git add .` → `git commit -m "feat(m3): agrega formulario de nueva solicitud"`
5	Sube la rama	`git push -u origin feat/m3-formulario-solicitud` (después, solo `git push`)
6	Abre el pull request hacia main con la plantilla: qué cambia, cómo probarlo, capturas y checklist. Escribe «Cierra #N» para enlazar la tarea	GitHub
7	Revisión: al menos un compañero. Si toca base de datos, autenticación o archivos compartidos, revisa Diego. Corrige con nuevos commits en la misma rama	GitHub
8	Fusión: la hace Diego con «Create a merge commit» (por la restricción de Vercel Hobby); luego se borra la rama	GitHub
9	Actualiza tu rama con main cada día	`git switch main` → `git pull` → `git switch tu-rama` → `git merge main`
10	Si hay conflicto, VS Code muestra las opciones (cambio actual, entrante o ambos). Si no entiendes el otro cambio, pregúntale a su autor. Luego guarda y haz commit	`git add .` → `git commit`

- Nunca hagas commits directos a main ni `git push --force`.
- Pull requests de menos de un día de trabajo: son más fáciles de revisar y casi no generan conflictos.
- No subas `.env.local`, `node_modules` ni claves. Si `git status` los muestra, detente y pregunta.

D. Convenciones de base de datos

- **Dos proyectos de Supabase** [SUPUESTO; se confirma en la Parte 3]: `offero-dev` para desarrollo y pruebas (ahí sí puedes crear datos de prueba) y `offero-prod` solo con datos reales (lo usa la versión de producción en Vercel). Tu `.env.local` y las previews apuntan a dev.
- **Cada cambio de esquema es un archivo** en `supabase/sql/` con nombre `AAAAMMDD_HHMM_descripcion.sql` (por ejemplo `20261111_1530_requests.sql`). La fecha y hora evitan choques entre cuatro personas. Se prueba en dev, va en el PR y Diego lo aplica en prod al fusionar. Nunca edites un archivo ya aplicado: crea uno nuevo. Nadie cambia tablas desde el panel sin dejar su archivo.
- **Todas las tablas**: id uuid automático (profiles usa el id de Auth), created_at automático, claves foráneas explícitas y restricciones CHECK para estados y números.
- **RLS se activa en el mismo archivo que crea la tabla.** Una tabla sin políticas no se fusiona.
- **Contenido moderable** (businesses, requests, reviews) incluye `is_hidden` desde su creación, para que M7 no tenga que modificar tablas ajenas [SUPUESTO; registrar en la sección 11 del documento].
- **Funciones de ayuda de Diego**: `is_admin()` y `get_my_role()`. Úsenlas en las políticas en vez de repetir consultas.

- **Nombres de personas**: siempre desde la vista `public_profiles`. Los correos nunca se muestran a otros usuarios.
- **Validación en tres capas**: formulario (comodidad), servidor de Next.js (seguridad) y base de datos (restricciones y RLS, la última barrera).

Tabla.campo	Valores acordados (fuente única)
`profiles.role`	client, provider, admin
`requests.status`	open (recibe cotizaciones), awarded (adjudicada), cancelled
`quotes.status`	pending, accepted, rejected, withdrawn
`reports.status`	open, reviewed, dismissed
`reports.target_type`	business, request, review

E. Archivos compartidos

Archivo o carpeta	Dueño	Regla
`src/app/layout.js`, `globals.css`, `src/components/layout/`	Daniel	Avisa en Discord antes de cambiarlos; cambios pequeños
`src/lib/supabase/` y archivo de sesión (proxy o middleware)	Diego	Nadie crea otro cliente de Supabase
`supabase/sql/`	Dueño de cada tabla; aprueba Diego	No editar archivos ya aplicados
`package.json` y `package-lock.json`	Todos, con acuerdo	No instales paquetes sin acordarlo (restricción de stack). Si hay conflicto en el lock: acepta la versión de main y vuelve a ejecutar `npm install`
`docs/` (documento de contexto)	Cada uno, su módulo	Actualiza las secciones 8, 9 y 10 al avanzar
`src/components/ui/`	Quien crea el componente	Si cambias un componente compartido (ReportButton, CategoryFilter), avisa

F. Comunicación, regla de atasco e IA

- **Canales** [SUPUESTO]: Discord para el trabajo (#daily para el avance diario, #ayuda para bloqueos, #prs para pedir revisión). WhatsApp solo para urgencias.
- **Regla de atasco**: después de 45 minutos intentando solo, escribe en #ayuda qué intentabas, el archivo, el mensaje de error completo y qué probaste. Si en 2 horas nadie responde, menciona a Diego. Un bloqueo de más de un día se dice en la Daily.
- **Revisiones**: responde las que te pidan en menos de 24 horas.
- **IA**: pega el documento de contexto al iniciar cada conversación; pide explicaciones, no solo código; verifica que use JavaScript, App Router y `@supabase/ssr` (no TypeScript, ni la carpeta `pages/`, ni Flask); no subas nada que no puedas explicar en la revisión; nunca pegues claves en la IA.

G. Definition of Done

Una historia está hecha cuando:

- cumple sus criterios de aceptación;
- su pull request fue revisado y fusionado;
- funciona en la Preview y luego en producción;
- sus permisos se probaron con dos cuentas de roles distintos;

- no deja errores en la consola del navegador;
- el documento de contexto refleja cualquier cambio.

Un módulo está Terminado cuando (regla del documento de contexto):

- funciona en la versión publicada en Vercel;
- sus operaciones principales están probadas por otra persona (prueba cruzada);
- no depende de datos ficticios en producción;
- la sección 8 del documento dice Terminado y Publicado: Sí.